

Учреждение образования
«БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ»

Факультет информационных технологий и управления
Кафедра интеллектуальных информационных технологий

Отчёт по расчётной работе
“Представление и обработка информации в интеллектуальных системах”

Выполнил:

студент группы 521703

Проверил:

ассистент кафедры ИИТ
Зотов Н. В.

Минск, 2026

1. Цель работы

Приобретение практических навыков построения и программной реализации алгоритмов обработки графов и освоение различных способов представления графов в памяти вычислительной системы и сравнительный анализ эффективности выбранных представлений при решении прикладных задач.

2. Задание

Реализовать выбранный алгоритм на графах с использованием одного из способов представления структуры графа. Провести тестирование работы программы на различных входных данных, проанализировать сложность алгоритма и эффективность выбранной структуры данных.

3. Варианты

1. Вариант представления графа: 4
2. Вариант алгоритма: 9

4. Теоретические сведения

Для выполнения задания необходимо понимать задачу алгоритма:

Для заданной транспортной сети (ориентированный граф с пропускными способностями ребер, истоком S и стоком T) найти максимальную величину потока, который можно переслать из S в T , не превышая пропускные способности рёбер.

Список инцидентности – это один из способов представления графа в памяти компьютера. Который для каждой вершины хранит список рёбер, которые с ней инцидентны

Алгоритм построения списка инцидентности:

Берётся граф, у которого есть вершины и рёбра. Для каждой вершины заводится отдельный список. Когда нужно добавить ребро, которое идет, например, из вершины A в вершину B , то в список вершины A записывается информация об этом ребре: куда оно ведет (в вершину B) и какая у него пропускная способность. Так делается для каждого ребра графа. В результате у каждой вершины получается свой список всех рёбер, которые из нее выходят. Если граф ориентированный, то для вершины B в её списке это ребро не появится, потому что ребро идет из A в B , а не наоборот. Если же граф неориентированный, то ребро записывается в списки обеих вершин. В случае алгоритма Диница дополнительно добавляются еще и обратные рёбра с нулевой пропускной способностью, чтобы алгоритм мог корректно перераспределять поток, но это уже особенность конкретного алгоритма, а не самого списка инцидентности.

Алгоритм Диница решает задачу поиска максимального потока в ориентированном графе. Он работает итеративно, каждая итерация состоит из двух этапов.

На первом этапе строится уровневый граф с помощью обхода в ширину. Каждой вершине присваивается расстояние от истока. При обходе учитываются только рёбра с положительной остаточной пропускной способностью. Если сток недостижим, алгоритм завершается.

На втором этапе находится блокирующий поток с помощью обхода в глубину. Движение происходит строго по возрастанию уровней. Каждый найденный путь даёт величину потока, равную минимальной пропускной способности на пути. Пропускные способности прямых рёбер уменьшаются, обратных — увеличиваются. Поиск продолжается, пока существуют пути.

После завершения второго этапа алгоритм переходит к следующей итерации — перестраивает уровневый граф с учётом новых остаточных пропускных способностей. Алгоритм заканчивается, когда сток становится недостижим.

5. Листинг программы

```
from collections import deque
import sys
import matplotlib.pyplot as plt
import networkx as nx

sys.setrecursionlimit(10**6)

class Edge:
    def __init__(self, to, rev, capacity):
        self.to = to
        self.rev = rev
        self.capacity = capacity

    def __repr__(self):
        return f"Edge(to={self.to}, capacity={self.capacity})"

class Dinic:
    def __init__(self, n):
        self.n = n
        # Реализация представления графа - список инцидентности
        self.graph = [[] for _ in range(n)]
        self.original_capacities = {}

    def add_edge(self, fr, to, cap):
        # Обработка кратных рёбер (пропускные способности суммируются)
```

```

        forward = Edge(to, len(self.graph[to]), cap)
        backward = Edge(fr, len(self.graph[fr]), 0)
        self.graph[fr].append(forward)
        self.graph[to].append(backward)
        self.original_capacities[(fr, to)] =
self.original_capacities.get((fr, to), 0) + cap

def bfs_levels(self, s, t):
    self.level = [-1] * self.n
    q = deque()
    self.level[s] = 0
    q.append(s)

    while q:
        v = q.popleft()
        for e in self.graph[v]:
            if e.capacity > 0 and self.level[e.to] < 0:
                self.level[e.to] = self.level[v] + 1
                q.append(e.to)
    return self.level[t] >= 0

def dfs_flow(self, v, t, f, it):
    if v == t:
        return f
    for i in range(it[v], len(self.graph[v])):
        it[v] = i
        e = self.graph[v][i]
        if e.capacity > 0 and self.level[v] < self.level[e.to]:
            ret = self.dfs_flow(e.to, t, min(f, e.capacity), it)
            if ret > 0:
                e.capacity -= ret
                self.graph[e.to][e.rev].capacity += ret
                return ret
    return 0

def max_flow(self, s, t):
    if s == t:
        return 0
    flow = 0
    INF = 10**18
    while self.bfs_levels(s, t):
        it = [0] * self.n
        while True:
            pushed = self.dfs_flow(s, t, INF, it)
            if pushed == 0:
                break
            flow += pushed

```

```

        return flow

def get_flow(self, s, t):
    flow_dict = {}
    for u in range(self.n):
        for e in self.graph[u]:
            if e.capacity < self.original_capacities.get((u, e.to), 0):
                flow = self.original_capacities.get((u, e.to), 0) -
e.capacity
            if flow > 0:
                flow_dict[(u, e.to)] = flow
    return flow_dict

# Вывод результатов в удобной форме - список инцидентности
def print_incidence_list(self):
    print("\n--- Список инцидентности (остаточная сеть) ---")
    for i, edges in enumerate(self.graph):
        print(f"Вершина {i}:")
        for e in edges:
            print(f"- {e.to} (cap: {e.capacity})")
    print("-----")

# Вывод результатов в удобной форме - визуализация графа
def visualize_graph(self, s, t, flow=None):
    G = nx.DiGraph()

    for i in range(self.n):
        G.add_node(i)

    edge_labels = {}
    for u in range(self.n):
        for e in self.graph[u]:
            if e.capacity > 0 and u != e.to:
                G.add_edge(u, e.to)
                orig_cap = self.original_capacities.get((u, e.to), 0)
                if flow and (u, e.to) in flow:
                    edge_labels[(u, e.to)] = f"{flow[(u,
e.to)]}/{orig_cap}"
                else:
                    edge_labels[(u, e.to)] = f"0/{orig_cap}"

    plt.figure(figsize=(12, 8))
    pos = nx.spring_layout(G, k=2, iterations=50)

    node_colors = []
    for node in G.nodes():
        if node == s:

```

```

        node_colors.append('lightgreen')
    elif node == t:
        node_colors.append('lightcoral')
    else:
        node_colors.append('lightblue')

    nx.draw_networkx_nodes(G, pos, node_color=node_colors,
node_size=500, alpha=0.8)
    nx.draw_networkx_edges(G, pos, edge_color='gray', arrows=True,
                           arrowsize=20, arrowstyle='->', width=2)
    nx.draw_networkx_labels(G, pos, font_size=12, font_weight='bold')
    nx.draw_networkx_edge_labels(G, pos, edge_labels, font_size=10)

    if flow:
        total_flow = sum(flow.values())
        plt.title(f"Максимальный поток: {total_flow} (Исток: {s} ->
Сток: {t})",
                fontsize=14, fontweight='bold')
    else:
        plt.title(f"Граф потока (Исток: {s} -> Сток: {t})",
                fontsize=14, fontweight='bold')

    from matplotlib.patches import Patch
    legend_elements = [
        Patch(facecolor='lightgreen', alpha=0.8, label='Исток'),
        Patch(facecolor='lightcoral', alpha=0.8, label='Сток'),
        Patch(facecolor='lightblue', alpha=0.8, label='Промежуточная
вершина')
    ]
    plt.legend(handles=legend_elements, loc='upper right')

    plt.axis('off')
    plt.tight_layout()
    plt.show()

def visualize_residual_network(self, s, t):
    G = nx.DiGraph()

    for i in range(self.n):
        G.add_node(i)

    edge_colors = []
    edge_labels = {}

    for u in range(self.n):
        for e in self.graph[u]:
            if e.capacity > 0:

```

```

        G.add_edge(u, e.to)
        edge_labels[(u, e.to)] = f"{e.capacity}"
        if (u, e.to) in self.original_capacities:
            edge_colors.append('blue')
        else:
            edge_colors.append('red')

plt.figure(figsize=(12, 8))
pos = nx.spring_layout(G, k=2, iterations=50)

node_colors = []
for node in G.nodes():
    if node == s:
        node_colors.append('lightgreen')
    elif node == t:
        node_colors.append('lightcoral')
    else:
        node_colors.append('lightblue')

nx.draw_networkx_nodes(G, pos, node_color=node_colors,
node_size=500, alpha=0.8)
nx.draw_networkx_edges(G, pos, edge_color=edge_colors, arrows=True,
                        arrowsize=20, arrowstyle='->', width=2)
nx.draw_networkx_labels(G, pos, font_size=12, font_weight='bold')
nx.draw_networkx_edge_labels(G, pos, edge_labels, font_size=9)

plt.title(f"Остаточная сеть (Синие - прямые рёбра, Красные -
обратные)",
        fontsize=12)
plt.axis('off')
plt.tight_layout()
plt.show()

# Загрузка исходных данных графа из файла
def load_graph_from_file(filename):
    try:
        with open(filename, 'r') as f:
            lines = f.read().strip().split('\n')
            if not lines:
                raise ValueError("Пустой файл")
            n, m = map(int, lines[0].split())
            s, t = map(int, lines[1].split())

            if len(lines) < 2 + m:
                raise ValueError(f"Недостаточно строк: ожидается {2+m},
получено {len(lines)}")

```

```

        dinic = Dinic(n)
        for i in range(2, 2 + m):
            u, v, c = map(int, lines[i].split())
            if u >= n or v >= n:
                raise ValueError(f"Вершина {u} или {v} вне диапазона 0..{n-1}")

            dinic.add_edge(u, v, c)
        return dinic, s, t
    except Exception as e:
        print(f"Ошибка при чтении файла: {e}")
        return None, None, None

# Загрузка исходных данных графа из консоли
def load_graph_from_console():
    try:
        n = int(input("Введите количество вершин: "))
        if n < 0:
            raise ValueError("Количество вершин не может быть отрицательным")

        m = int(input("Введите количество рёбер: "))
        s = int(input("Введите номер истока: "))
        t = int(input("Введите номер стока: "))

        if s >= n or t >= n:
            raise ValueError(f"Исток или сток вне диапазона 0..{n-1}")

        dinic = Dinic(n)
        print("Введите рёбра в формате: from to capacity")
        for i in range(m):
            u, v, c = map(int, input(f"Рёбро {i+1}: ").split())
            if u >= n or v >= n:
                raise ValueError(f"Вершина {u} или {v} вне диапазона 0..{n-1}")

            if c < 0:
                raise ValueError(f"Пропускная способность не может быть отрицательной: {c}")

            dinic.add_edge(u, v, c)
        return dinic, s, t
    except Exception as e:
        print(f"Ошибка ввода: {e}")
        return None, None, None

# Тестирование корректной обработки графов

```



```

def run_tests():
    # ПУНКТ 4a: Пустой граф
    print("\n" + "=" * 60)
    print("ТЕСТ 1: Пустой граф (0 вершин)")
    dinic_empty = Dinic(0)
    print(f"Пустой граф создан: n={dinic_empty.n}")

    # ПУНКТ 4b: Граф с одной вершиной
    print("\n" + "=" * 60)
    print("ТЕСТ 2: Граф с одной вершиной")
    dinic_one = Dinic(1)
    dinic_one.add_edge(0, 0, 100) # ПУНКТ 4c: Петля
    print(f"Поток из 0 в 0: {dinic_one.max_flow(0, 0)} (ожидается 0)")
    dinic_one.print_incidence_list()

    # Вывод результатов
    print("\n" + "=" * 60)
    print("ТЕСТ 3: Простой граф с двумя вершинами")
    dinic_two = Dinic(2)
    dinic_two.add_edge(0, 1, 5)
    flow = dinic_two.max_flow(0, 1)
    print(f"Поток из 0 в 1: {flow} (ожидается 5)")
    dinic_two.visualize_graph(0, 1, dinic_two.get_flow(0, 1))

    # Граф с кратными рёбрами
    print("\n" + "=" * 60)
    print("ТЕСТ 4: Граф с кратными рёбрами")
    dinic_multi = Dinic(2)
    dinic_multi.add_edge(0, 1, 3)
    dinic_multi.add_edge(0, 1, 4)
    flow = dinic_multi.max_flow(0, 1)
    print(f"Суммарный поток: {flow} (ожидается 7)")
    dinic_multi.visualize_graph(0, 1, dinic_multi.get_flow(0, 1))

def main():
    print("Алгоритм Диница (максимальный поток) — представление: список инцидентности")

    print("Выберите способ загрузки графа:")
    print("1. Загрузить из файла")
    print("2. Ввести с консоли")
    print("3. Запустить автоматические тесты")
    choice = input("Ваш выбор: ")

    if choice == "1":
        filename = input("Введите имя файла: ")
        dinic, s, t = load_graph_from_file(filename)

```

```

        if dinic:
            flow = dinic.max_flow(s, t)
            print(f"Максимальный поток из {s} в {t}: {flow}")
            dinic.print_incidence_list()

            show_viz = input("Показать визуализацию графа? (y/n): ").lower()
            if show_viz == 'y':
                dinic.visualize_graph(s, t, dinic.get_flow(s, t))
                show_residual = input("Показать остаточную сеть? (y/n): ").lower()

                if show_residual == 'y':
                    dinic.visualize_residual_network(s, t)
            else:
                print("Не удалось загрузить граф.")
        elif choice == "2":
            dinic, s, t = load_graph_from_console()
            if dinic:
                flow = dinic.max_flow(s, t)
                print(f"Максимальный поток из {s} в {t}: {flow}")
                dinic.print_incidence_list()

                show_viz = input("Показать визуализацию графа? (y/n): ").lower()
                if show_viz == 'y':
                    dinic.visualize_graph(s, t, dinic.get_flow(s, t))
                    show_residual = input("Показать остаточную сеть? (y/n): ").lower()

                    if show_residual == 'y':
                        dinic.visualize_residual_network(s, t)
            else:
                print("Ошибка ввода.")
        elif choice == "3":
            run_tests()
        else:
            print("Неверный выбор, запускаю тесты по умолчанию.")
            run_tests()

if __name__ == "__main__":
    main()

```

6. Примеры входных данных и результаты работы программы

Пример 1(Ввод с консоли):

Выберите способ загрузки графа:

1. Загрузить из файла
2. Ввести с консоли
3. Запустить автоматические тесты

Ваш выбор: 2

Введите количество вершин: 5

Введите количество рёбер: 7

Введите номер истока: 0

Введите номер стока: 1

Введите рёбра в формате: from to capacity

Рёбро 1: 0 4 20

Рёбро 2: 4 3 50

Рёбро 3: 3 0 33

Рёбро 4: 3 2 12

Рёбро 5: 2 4 100

Рёбро 6: 2 1 55

Рёбро 7: 1 3 12

Максимальный поток из 0 в 1: 12

--- Список инцидентности (остаточная сеть) ---

Вершина 0:

- 4 (cap: 8)
- 3 (cap: 0)

Вершина 1:

- 2 (cap: 12)
- 3 (cap: 12)

Вершина 2:

- 3 (cap: 12)
- 4 (cap: 100)
- 1 (cap: 43)

Вершина 3:

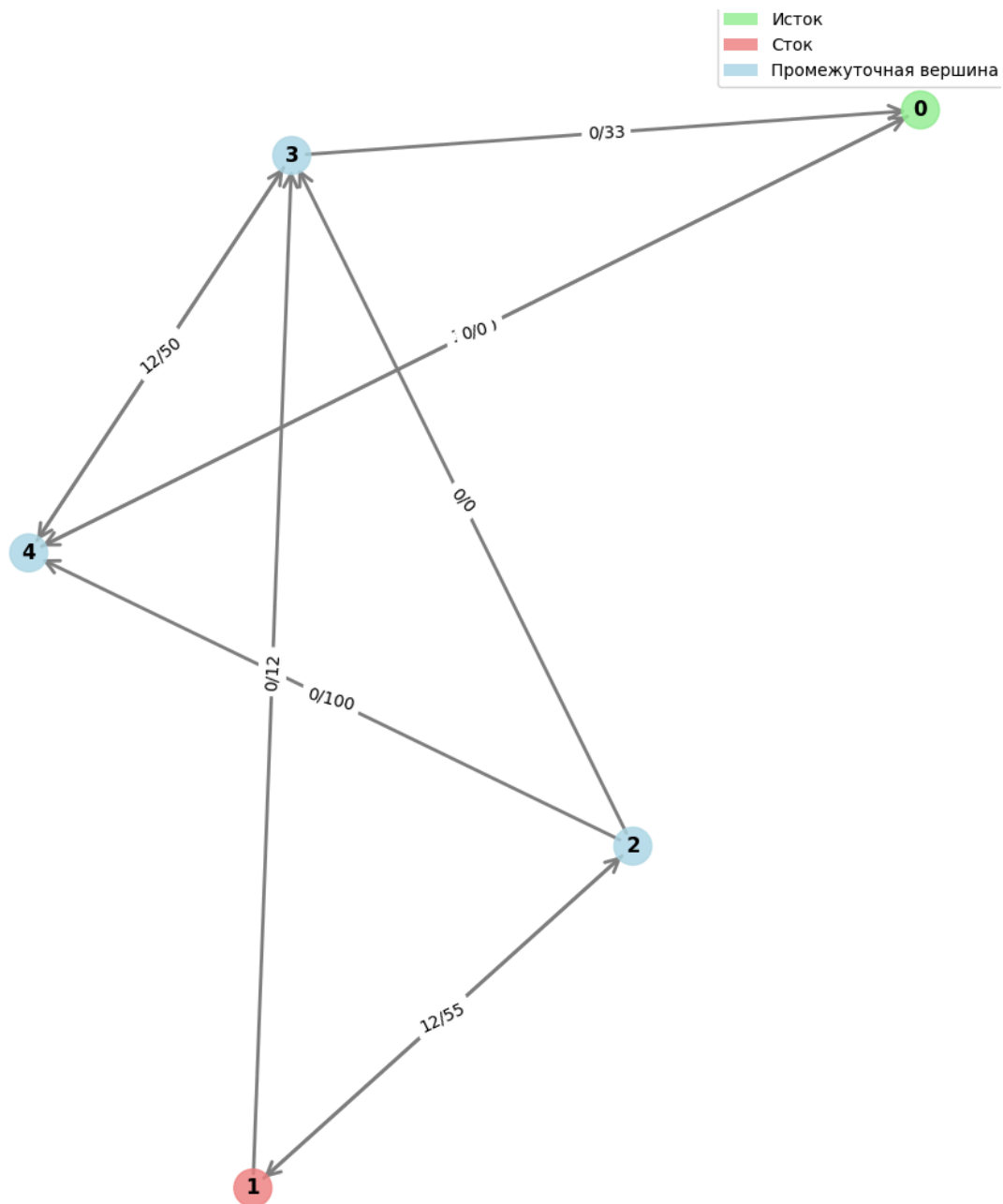
- 4 (cap: 12)
- 0 (cap: 33)
- 2 (cap: 0)
- 1 (cap: 0)

Вершина 4:

- 0 (cap: 12)
- 3 (cap: 38)
- 2 (cap: 0)

Показать визуализацию графа? (y/n): y

☐



Пример 2 (Загрузка из файла):

Содержимое файла

```

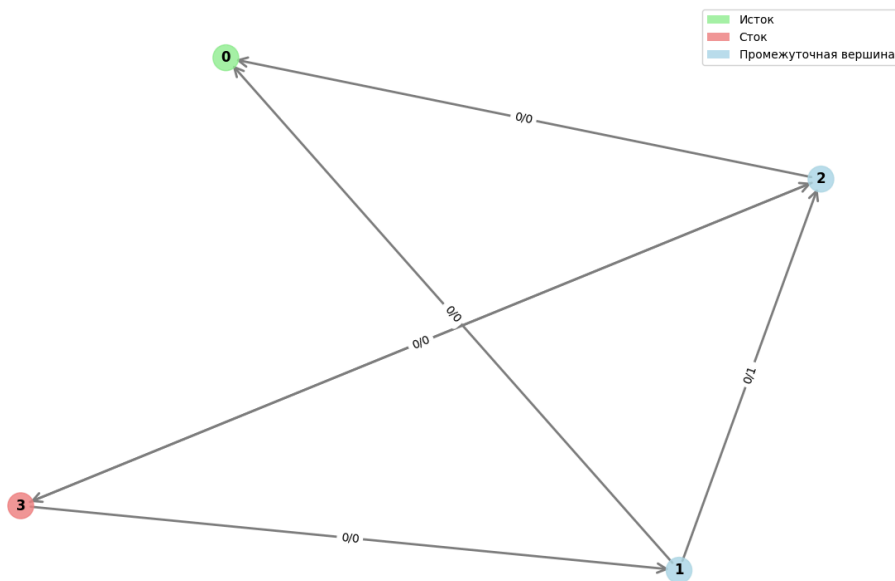
test_graph.txt
1  4 5
2  0 3
3  0 1 2
4  0 2 3
5  1 2 1
6  1 3 2
7  2 3 4
8
  
```

```

Выберите способ загрузки графа:
1. Загрузить из файла
2. Ввести с консоли
3. Запустить автоматические тесты
Ваш выбор: 1
Введите имя файла: test_graph.txt
Максимальный поток из 0 в 3: 5

--- Список инцидентности (остаточная сеть) ---
Вершина 0:
- 1 (сар: 0)
- 2 (сар: 0)
Вершина 1:
- 0 (сар: 2)
- 2 (сар: 1)
- 3 (сар: 0)
Вершина 2:
- 0 (сар: 3)
- 1 (сар: 0)
- 3 (сар: 1)
Вершина 3:
- 1 (сар: 2)
- 2 (сар: 3)
-----
Показать визуализацию графа? (y/n): y

```



7. Выводы

В ходе работы был реализован алгоритм Диница для нахождения максимального потока в ориентированном графе. Использовано представление графа в виде списка инцидентности с добавлением обратных рёбер. Реализована загрузка данных из файла и с консоли, а также визуализация графа и остаточной сети.

Цель работы — приобретение практических навыков реализации алгоритмов обработки графов — достигнута.

Практические выводы:

- i. Особенности алгоритма и представления графа:

Список инцидентности обеспечивает эффективное добавление ребер и модификацию остаточных пропускных способностей за постоянное время. Комбинация обхода в ширину для построения уровня графа и обхода в глубину для поиска блокирующего потока дает вычислительную сложность $O(E \cdot V^2)$, что является приемлемым для большинства практических задач. Наличие перекрёстных ссылок между прямыми и обратными ребрами позволяет обновлять пропускные способности за $O(1)$ без необходимости поиска обратного ребра.

ii. Ограничения/недостатки реализации:

При количестве вершин более тысячи рекурсивная реализация обхода в глубину может вызвать переполнение стека вызовов. Петли в графе корректно обрабатываются, однако не оказывают влияния на величину максимального потока, поскольку поток через петлю не может увеличить пропускную способность между истоком и стоком. Кроме того, визуализация с использованием алгоритма пружинной раскладки становится нечитаемой при количестве вершин свыше пятидесяти.

iii. В текущей реализации можно заменить рекурсивный обход в глубину на итеративный, чтобы избежать переполнения стека на больших графах.

8. Список использованных источников

1. Диниц Е.А. Алгоритм решения задачи о максимальном потоке в сети со степенной оценкой
2. Кормен Т., Лейзерсон Ч., Ривест Р., Штайн К. Алгоритмы: построение и анализ.
3. Бурсиан Е.Ю., Дёмин А.М. Основы теории графов: учебное пособие. – СПб.: ПГУПС, 2022.
4. Кристофидес Н. Теория графов. Алгоритмический подход.